FPT Education

TRƯỜNG ĐẠI HỌC FPT

Module 8 – Working with RESTful APIs & JSON in Flutter

Learning Objectives

Content:

- Understand basic HTTP and RESTful API concepts
- Send GET requests in Flutter using the http package
- Parse JSON responses into Dart model classes
- Display API data using FutureBuilder + ListView
- Handle loading, empty, and error states
- Send POST requests to create or submit data
- Organize API logic using a reusable Service Layer pattern

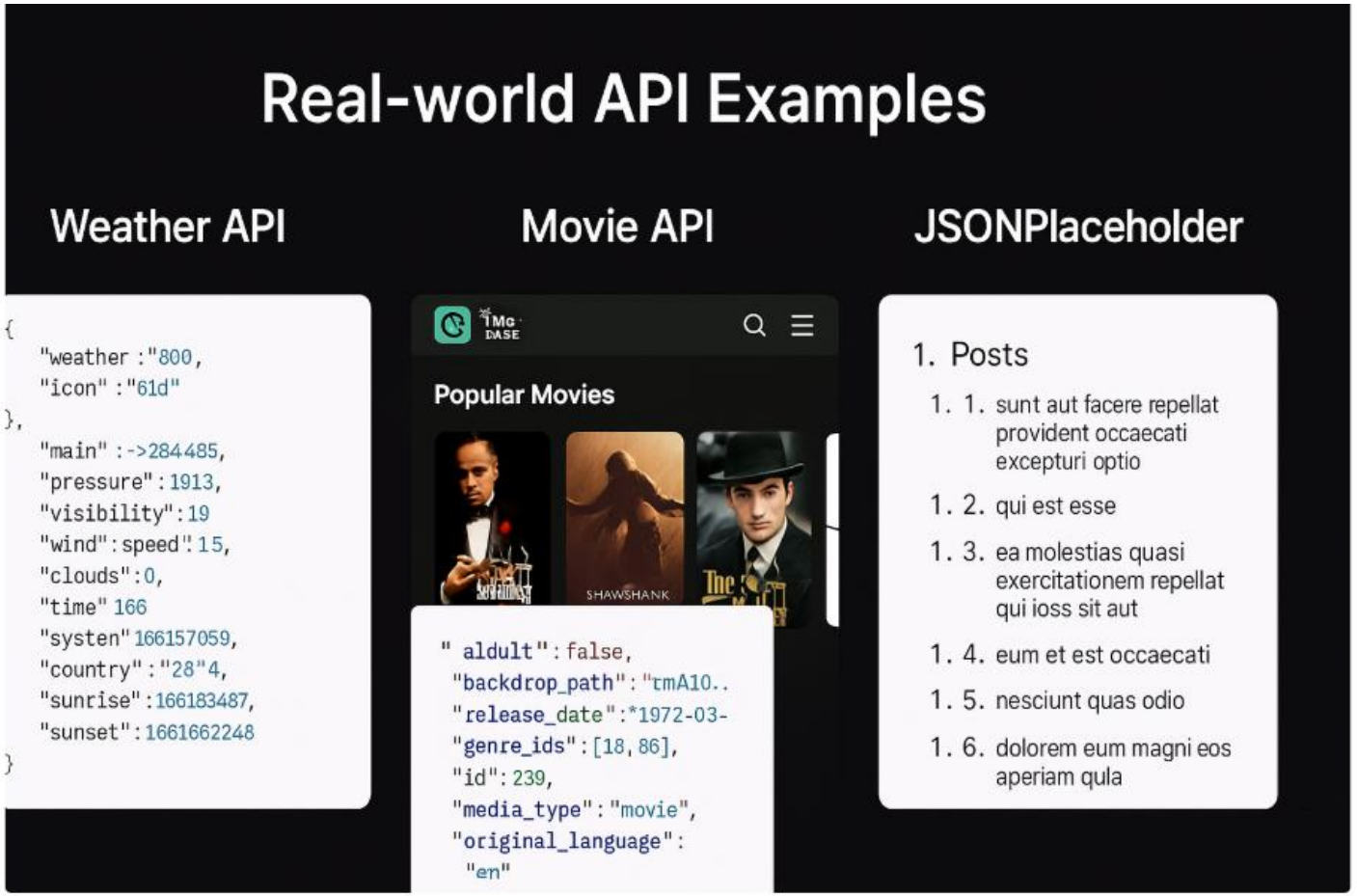
What is an API? What is HTTP?

API (Application Programming Interface)

- A contract that allows clients to talk to a server
- Often exposed as a set of URLs (endpoints)

HTTP fundamentals

- Methods: GET, POST, PUT, DELETE
- Status codes:
- 200–299: success
- 400–499: client error
- 500–599: server error



What is JSON?

JSON (JavaScript Object Notation)

- A lightweight text format for data exchange
- Key–value pairs and arrays
- Easy to read and parse
- Maps naturally to Dart structures: Map<String, dynamic> and List<dynamic>

- **JSON in APIs**
  - APIs return JSON → your Flutter app decodes → maps to Model classes.

JSON Structure

```
{
  "id": 1,
  "name": "Aliice",
  "email":"example.com"
}
```

A single JSON object (key-value pairs)

```
[
  { id" 1, "title":
  { title: "Post B"}
]
```

An array containing multiple JSON obj

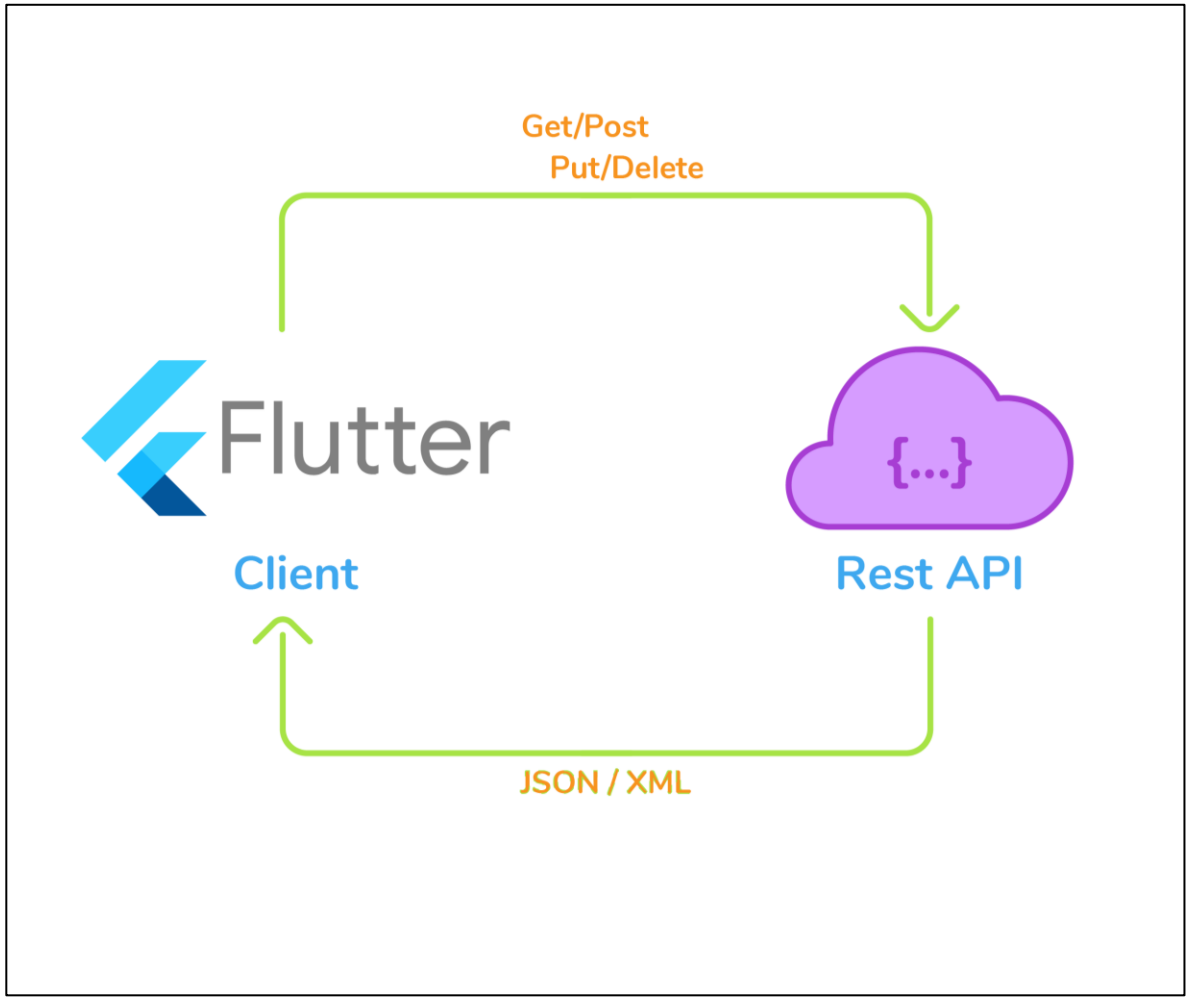
```
{
  "user": {
    "id" 1, title"Hello")
    "id" 2, title"Hi again")
}
```

Objects and arrays nested inside another object

API Networking in Flutter — The Big Picture

Steps to call an API in Flutter

1. Choose an endpoint (URL + method)
2. Send the request using http client
3. Receive JSON response
4. Decode JSON → Dart models
5. Update UI with the data
6. Handle loading & error states



Using the http Package

Common approach in Flutter

- Add dependency in pubspec.yaml: http
- Import in Dart file:

```
import 'package:http/http.dart' as http;
import 'dart:convert';
```

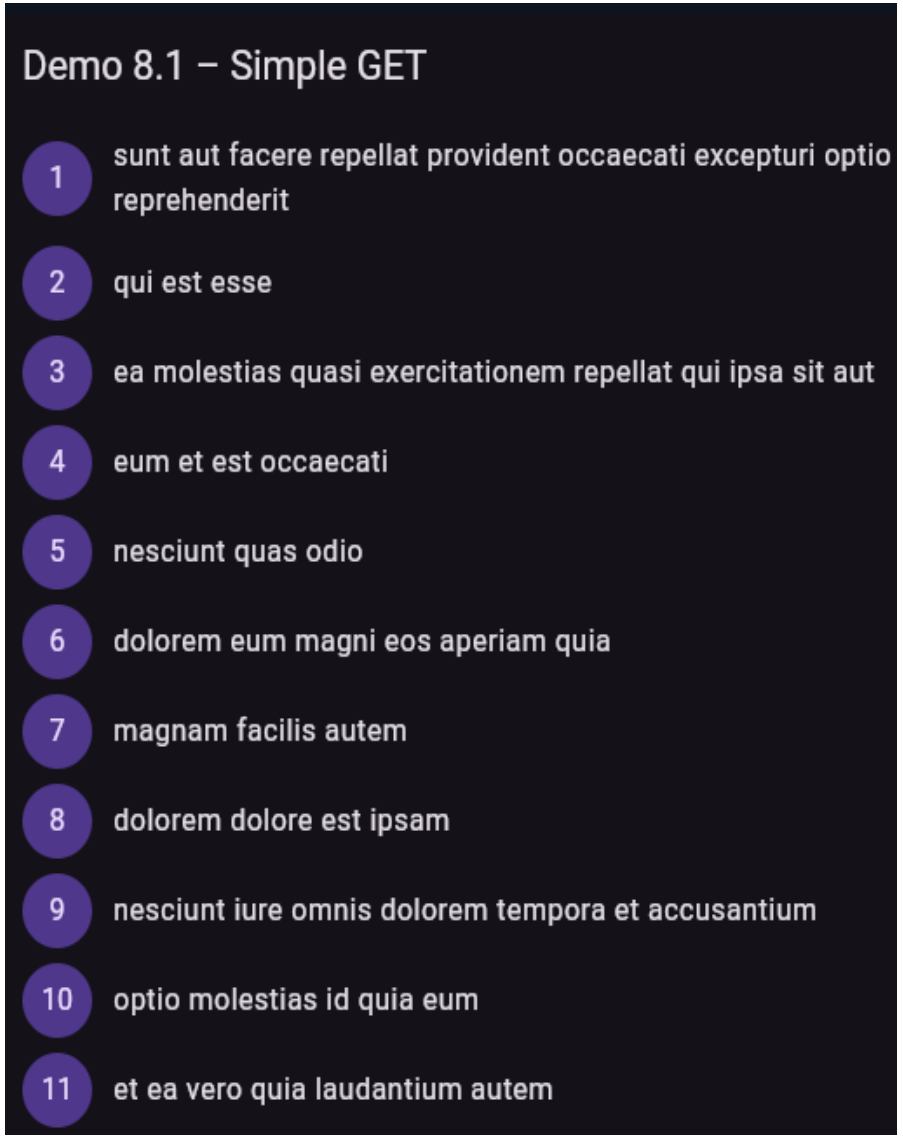
Typical usage

- http.get(Uri.parse(url))
- http.post(Uri.parse(url), body: {...})

Demo 8.1: Simple GET Request

Goal: Fetch data from a public REST API using an HTTP GET request and print the response.

- This demo demonstrates how to make a basic GET request in Flutter using the http package.
- The app will fetch a list of posts from the public API (https://jsonplaceholder.typicode.com/posts) and display the title of the first item.
- Steps to follow
  - Send an asynchronous HTTP GET request
  - Decode JSON into Dart objects
  - Handle loading status
  - Update UI when data arrives



Demo 8.1: Simple GET Request

```dart
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

void main() => runApp(const Demo81App());

class Demo81App extends StatelessWidget {
  const Demo81App({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Demo 8.1 - Simple GET',
      theme: ThemeData.dark(useMaterial3: true),
      home: const SimpleGetScreen(),
      debugShowCheckedModeBanner: false,
    );
  }
}

class Post {
  final int id;
  final String title;

  Post({required this.id, required this.title});

  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      id: json['id'] as int,
      title: json['title'] as String,
    );
  }
}

class SimpleGetScreen extends StatefulWidget {
  const SimpleGetScreen({super.key});

  @override
  State<SimpleGetScreen> createState() => _SimpleGetScreenState();
}
class _SimpleGetScreenState extends State<SimpleGetScreen> {
  late Future<List<Post>> _futurePosts;
  @override
  void initState() {
    super.initState();
    _futurePosts = fetchPosts();
  }
```

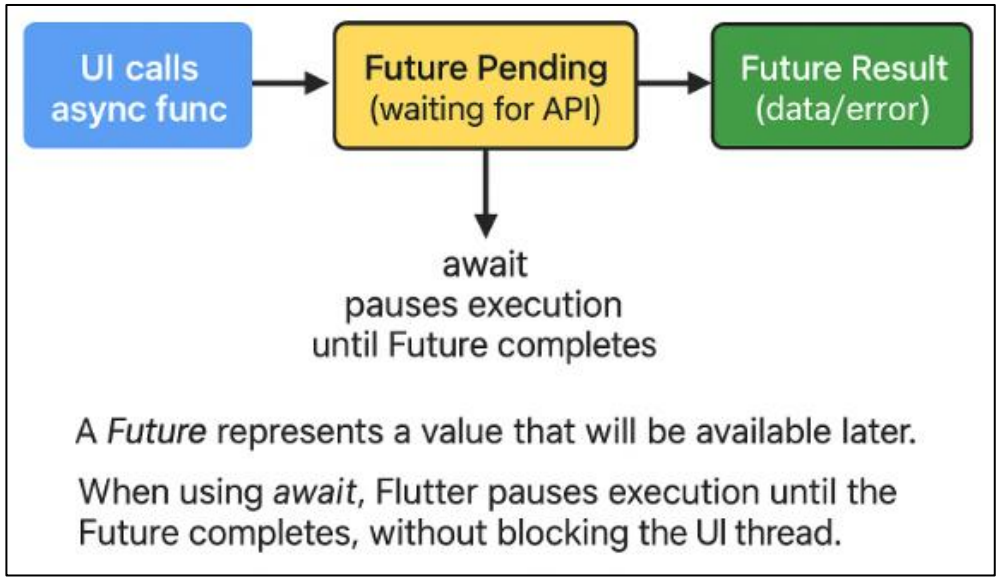
```dart
  Future<List<Post>> fetchPosts() async {
    final url = Uri.parse('https://jsonplaceholder.typicode.com/posts');
    final response = await http.get(url);

    if (response.statusCode != 200) {
      throw Exception('Failed to load posts');
    }
    final List<dynamic> jsonList = json.decode(response.body);
    return jsonList.map((item) => Post.fromJson(item)).toList();
  }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Demo 8.1 - Simple GET')),
      body: FutureBuilder<List<Post>>(
        future: _futurePosts,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          } else if (snapshot.hasError) {
            return Center(
              child: Text('Error: ${snapshot.error}'),
            );
          } else if (!snapshot.hasData || snapshot.data!.isEmpty) {
            return const Center(child: Text('No posts found'));
          }
          final posts = snapshot.data!;
          return ListView.builder(
            itemCount: posts.length,
            itemBuilder: (context, index) {
              final post = posts[index];
              return ListTile(
                title: Text(post.title),
                leading: CircleAvatar(child: Text(post.id.toString())),
              );
            },
          );
        },
      ),
    );
  }
}
```

Futures & async/await (Quick Review)

Key Concepts

- Future<T> represents a value that will be available later
- async marks a function as asynchronous
- await pauses execution until a Future completes
- Exam:

```
Future<void> fetchData() async {
  final result = await getDataFromServer();
  print(result);
}
```



Why async matters for Networking?

- Prevents UI freezing while waiting for network responses
- Lets Flutter handle long-running tasks efficiently
- Required for all HTTP operations

FutureBuilder: Binding Data to UI

Why FutureBuilder?

- Integrates async calls into the widget tree
- Handles loading, success, and error states in one place

This FutureBuilder listens to fetchPosts() and rebuilds the UI based on the Future state:

- waiting: show a loading spinner
- error: display the error message
- no data: show a “No data” message
- data loaded: build a ListView of post titles

```
FutureBuilder<List<Post>>(
  future: fetchPosts(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const Center(child: CircularProgressIndicator());
    } else if (snapshot.hasError) {
      return Center(child: Text('Error: ${snapshot.error}'));
    } else if (!snapshot.hasData || snapshot.data!.isEmpty) {
      return const Center(child: Text('No data'));
    }
    final posts = snapshot.data!;
    return ListView.builder(
      itemCount: posts.length,
      itemBuilder: (_, index) => Text(posts[index].title),
    );
  },
);
```

Parsing JSON into Dart Models

- **Why use model classes?**
  - Type safety
  - Clear structure
  - Easier to maintain and reuse

Example **model**:

```
class Post {
  final int id;
  final String title;
  final String body;

  Post({
    required this.id,
    required this.title,
    required this.body,
  });

  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      id: json['id'] as int,
      title: json['title'] as String,
      body: json['body'] as String,
    );
  }
}
```

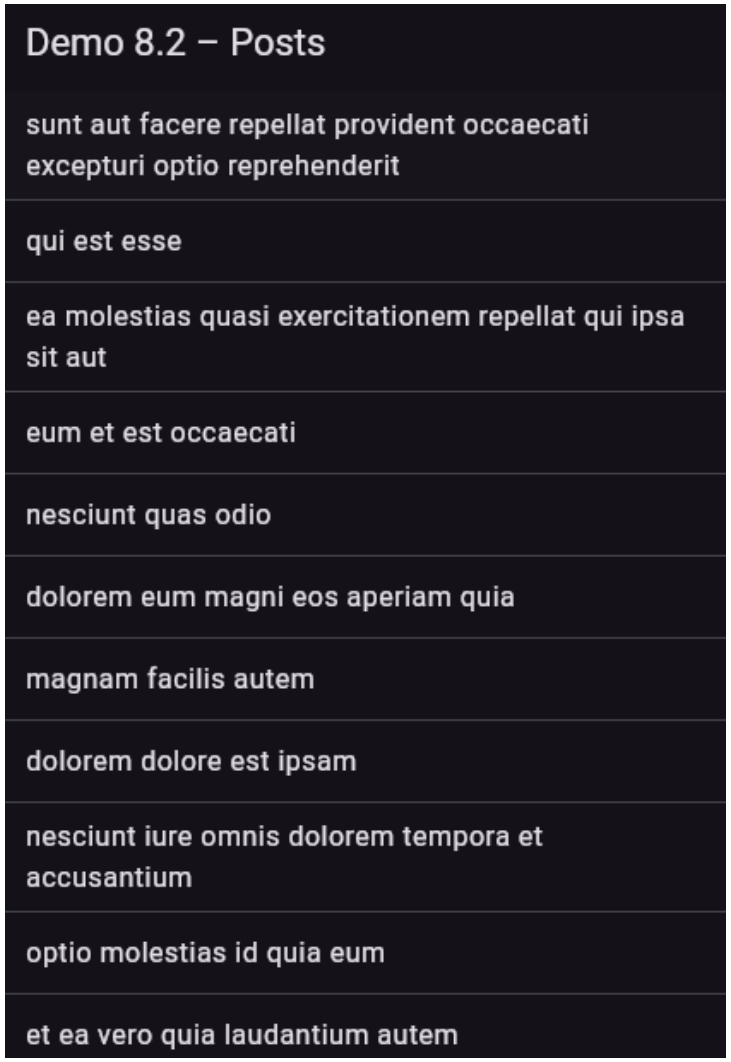
Demo 8.2 – JsonModelScreen

Objective

- Convert JSON into a Dart model
- Display list items and navigate to a Detail Screen with full data

Steps

- Create a model with fromJson
- Fetch and decode JSON
- Show list of items
- Tap an item → open Detail Screen
- Display selected item's fields



Converting JSON List to List<Model>

- **Key Idea**
  - Convert a JSON array (list of objects) into a typed List<Model> using decoding + mapping.
- **JSON → Dart Conversion**

```
final List<dynamic> jsonList = jsonDecode(response.body);

final posts = jsonList
    .map((e) => Post.fromJson(e))
    .toList();     // List<Post>
```

Concepts

- JSON list = array of multiple objects
- jsonDecode() → returns List<dynamic>
- .map() → converts each JSON item to a Model
- .toList() → final typed list (List<Post>)

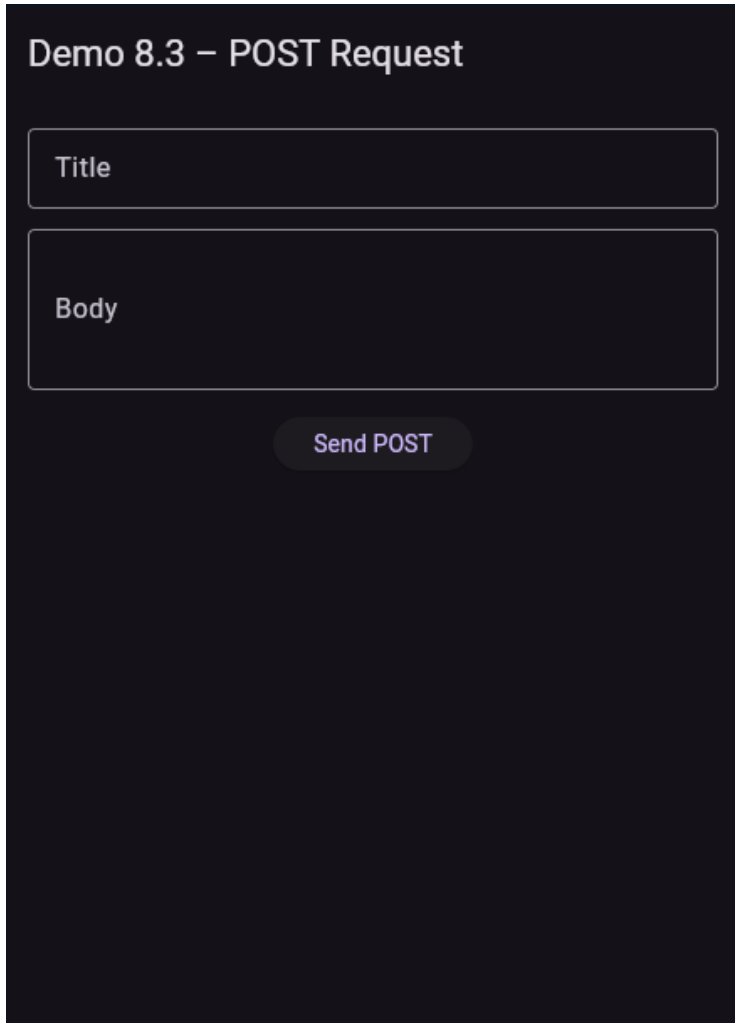
Demo 8.3: Sending Data via POST

Objectives

- Send data to an API using POST
- Show loading, success, and error states
- Allow user to retry on failure

Steps

1. Build a simple input form
2. Send POST request (http.post)
3. Encode JSON body
4. Show loading indicator
5. Handle error → show message + Retry button
6. Display success response



Error Handling & Retry

- **Common failure cases**
  - No internet connection
  - Server error (500)
  - Invalid JSON
  - Timeout
- **Basic strategies**
  - Throw exceptions in fetch functions
  - Show friendly error messages in UI
  - Offer a “Retry” button

Simple UI example:

```
if (snapshot.hasError) {
  return Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      Text('Error: ${snapshot.error}'),
      const SizedBox(height: 8),
      ElevatedButton(
        onPressed: () => setState(() {}),
        child: const Text('Retry'),
      ),
    ],
  );
}
```

POST Request + Error Handling Pattern

Explanation

- This block handles errors during the POST request.
- If the FutureBuilder detects an error (snapshot.hasError), it shows:
  - The error message
  - A Retry button that triggers setState() to run the request again
- It provides simple error feedback and a way for the user to retry the API call.

Basic pattern:

```
Future<void> createPost(String title, String body) async {
  final url = Uri.parse('https://jsonplaceholder.typicode.com/posts');
  final response = await http.post(
    url,
    headers: {'Content-Type': 'application/json; charset=UTF-8'},
    body: json.encode({
      'title': title,
      'body': body,
      'userId': 1,
    }),
  );

  if (response.statusCode == 201) {
    print('Post created: ${response.body}');
  } else {
    throw Exception('Failed to create post');
  }
}
```

Loading & Button States (UX)

Good UX patterns

- Disable submit button while request is in progress
- Show a loading indicator (spinner)
- Prevent double-submit

Key idea:

- Avoid double submit while loading

```
ElevatedButton(
  onPressed: isSubmitting ? null : _submit,
  child: isSubmitting
      ? const SizedBox(
          width: 16,
          height: 16,
          child: CircularProgressIndicator(strokeWidth: 2),
        )
      : const Text('Submit'),
);
```

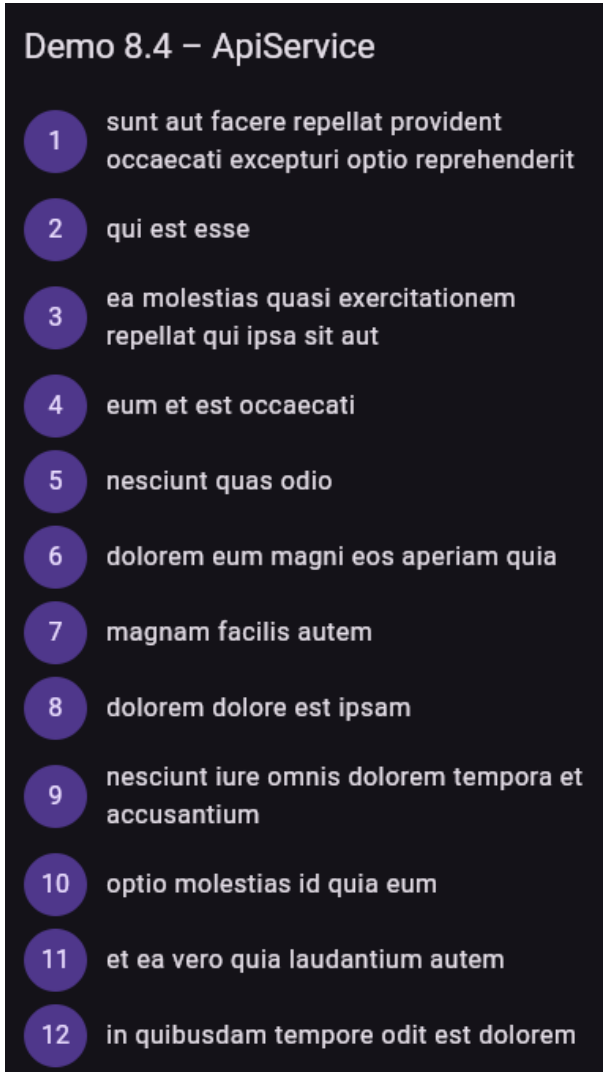
Demo 8.4 – ApiService

Objectives

- Move HTTP logic into a separate service
- Keep UI clean and focused
- Reuse the same service across screens

Steps

1. Create Post model
2. Implement ApiService.fetchPosts()
3. Call service in initState()
4. Use FutureBuilder to load + show list



Organizing Networking Code (Demo 8.4)

Service Layer Pattern

- Create a class to encapsulate API logic
- Keep widget code clean and focused on UI

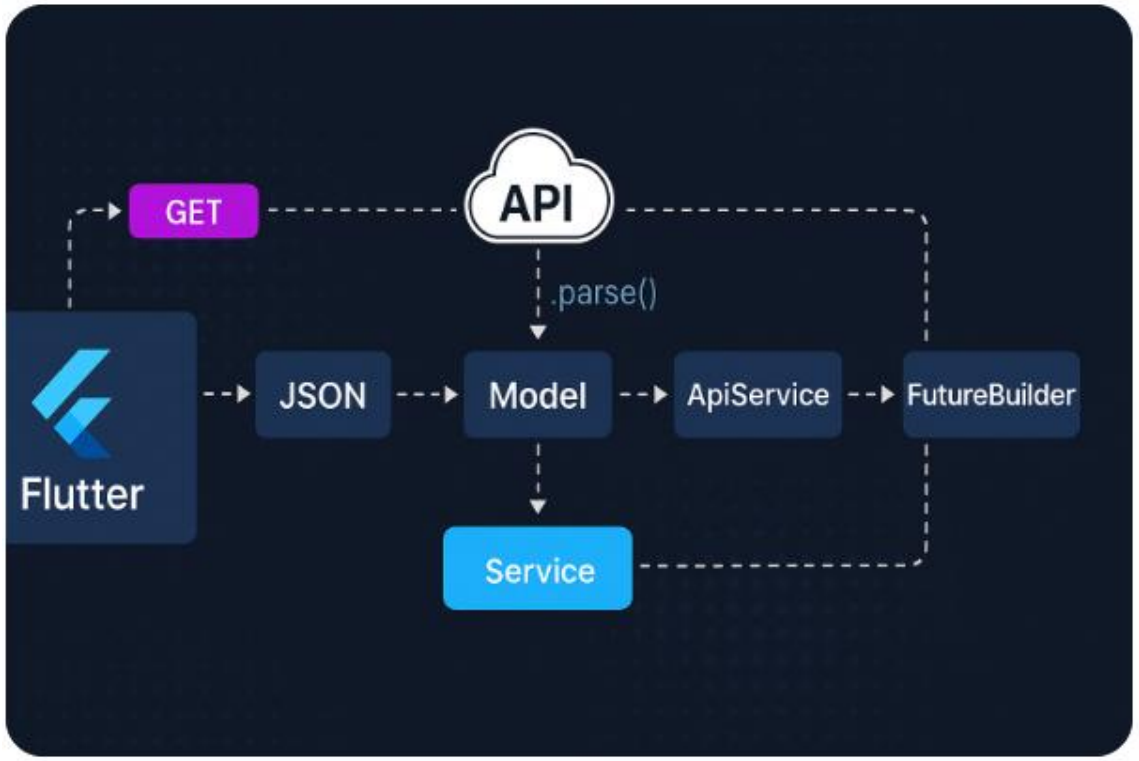
Example:

```
class ApiService {
  final http.Client client;

  ApiService({http.Client? client}) : client = client ?? http.Client();

  Future<List<Post>> fetchPosts() async {
    final url = Uri.parse('https://jsonplaceholder.typicode.com/posts');
    final response = await client.get(url);
    // ...parse & return list of Post
  }

  Future<Post> createPost(String title, String body) async {
    // ...call POST and return created Post
  }
}
```

From API to UI: Putting It All Together

End-to-end flow

- User opens screen
- App calls ApiService.fetchPosts()
- FutureBuilder shows loading → data / error
- User taps “Add” and opens form
- Form submits createPost()
- On success: show message, update list (re-fetch or optimistic update)



Custom Flutter API Pipeline Diagram PNG

Common Pitfalls & Best Practices

Pitfalls

- Calling APIs in build() instead of initState()
- Not handling errors → app crashes
- Blocking the UI thread with heavy parsing
- Hardcoding URLs everywhere

Best Practices

- Call API once in initState()
- Centralize base URL and endpoints
- Wrap networking in service classes):
- Log responses during development

Lab 8 Overview

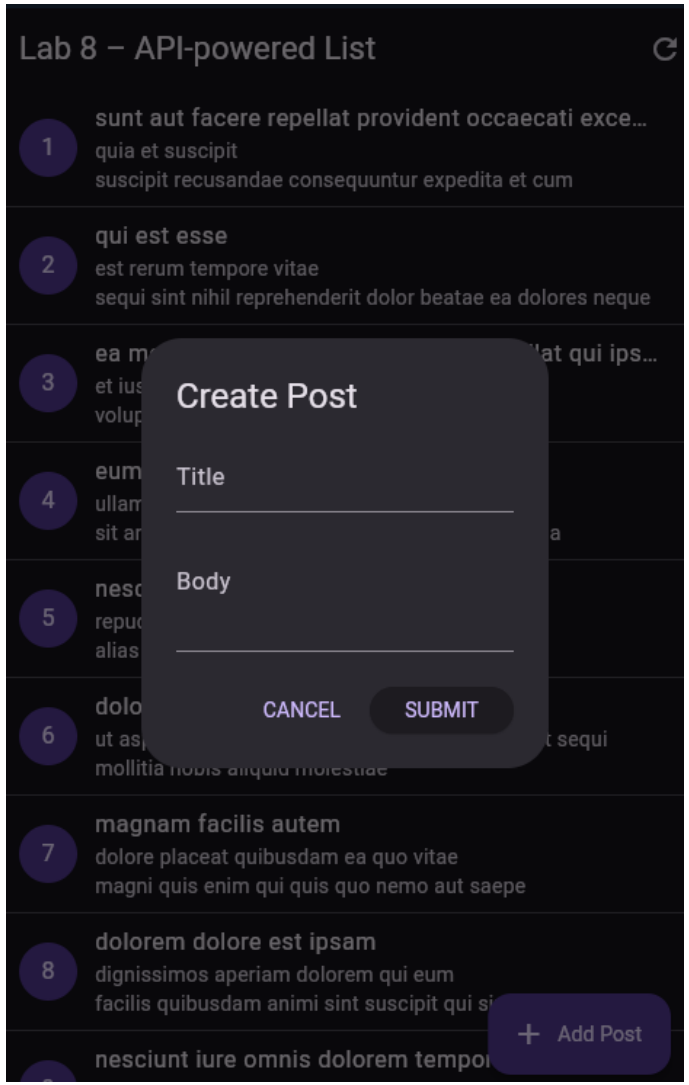
Objectives: Building a Simple API-powered List Screen

You will:

1. Call a public REST API
2. Parse JSON into model classes
3. Display a list of items using ListView
4. Show loading + error states

(Optional) Add a simple POST form to create a new item

Note: *This lab combines all concepts from Demos 8.1 → 8.4*



Summary

In this module, you learn how to:

- Understand how RESTful APIs work
- Send HTTP GET and POST requests in Flutter
- Decode JSON strongly into typed Dart model classes
- Display API data with FutureBuilder and ListView
- Implement loading, empty, and error UI states
- Organize networking code into a clean, reusable service layer

(Optional): Build a simple form that sends data via POST

References

- Mastering Flutter 2025 – Ch.3, 4, 9
- docs.flutter.dev
- **https://dartpad.dev**